|사물인터넷 실습|

Internet of Things and Laboratory

아두이노

강원대학교 컴퓨터공학과 권용훈

아두이노 (1/2)

아두이노의 기원

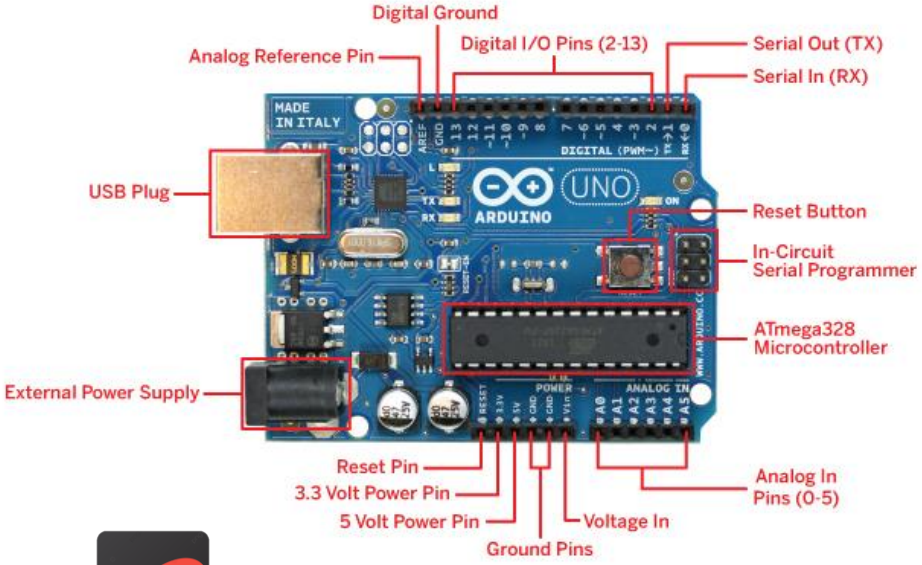
- 아두이노(Arduino): 오픈 소스 기반의 마이크로 컨트롤러를 위한 개발 보드
- 이탈리아의 IDII(Interaction Design Institute Ivrea)에서 디자인에 적용할 수 있는 상호 작용이 가능한 전자 장치를 쉽고 간편하게 만들 수 있도록 하기 위한 목적에서 시작
- 베이직 스탬프의 비싼 가격과 제한된 성능이 발단
  - 마시모 반지(Massimo Banzi): 아두이노 프로젝트 기획
  - 벤 프라이(Ben Fry)와 케이시 리아스(Casey Reas): 프로세싱(processing) 언어 개발
  - 에르난도 바라간(Hernando Barragan): 아두이노의 전신인 와이어링(wiring) 개발
- 와이어링의 구조를 단순화하여 $30 수준에서 아두이노를 개발
- 아두이노의 이름은 이탈리아 이브레아(Ivrea) 출신의 왕인 'Arduino d'Ivrea' 에서 착안

Arduino d'Ivrea
(Civica raccolta stampe - Milano)

아두이노 (2/2)

아두이노(UNO)의 성능 및 구조

- 마이크로 컨트롤러: ATmega328
  - Clock Frequency: 16MHz
  - 32개의 범용 레지스터(General Purpose Register)
- 메모리와 저장장치
  - Flash: 32KB
  - SRAM: 2KB
  - EEPROM: 1KB
- 동작 전압: 5V
- 입력 전압: 6~12V
- 디지털 입출력핀: 14개
- 아날로그 입력핀: 6개



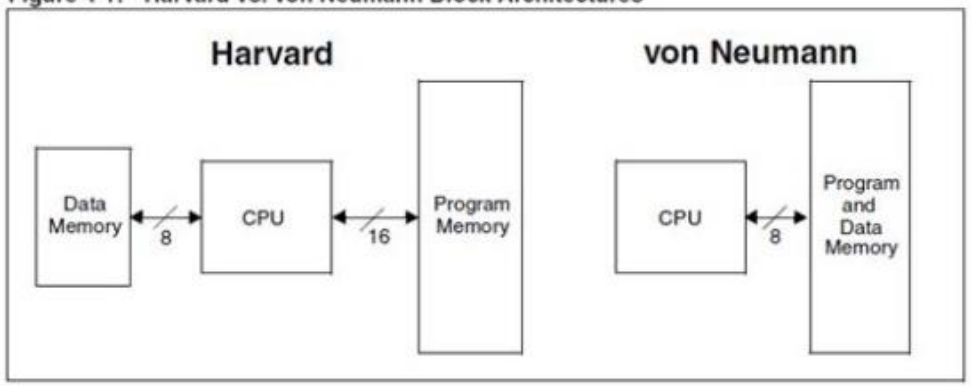

```
+-----------------+
|   FLASH (32KB)  | ← 프로그램 코드 저장
+-----------------+
|   EEPROM (1KB)  | ← 전원 OFF 후에도 데이터 유지
+-----------------+
|    SRAM (2KB)   | ← 변수 저장, 휘발성
+-----------------+
|    CPU (ALU)    | ← 연산 수행
+-----------------+
| General Purpose |
|  Registers (32) | ← 명령어 실행 시 데이터 저장
+-----------------+
| Interrupt System|
+-----------------+
|   Timer / PWM   | ← 정밀한 시간 제어
+-----------------+
|   ADC (10-bit)  | ← 아날로그 데이터 변환
+-----------------+
| UART / SPI / I2C | ← 통신 인터페이스
+-----------------+
|    GPIO (I/O)   | ← 디지털/아날로그 핀
+-----------------+
|   Clock System  | ← 16MHz 클럭
+-----------------+
```

아두이노 구성 (1/4)

마이크로 컨트롤러(Micro Controller)

- 마이크로 컨트롤러: 하나의 칩에 중앙처리장치(CPU) 뿐만 아니라 일정 용량의 메모리와 입출력 인터페이스 까지 내장한 특수 목적용 마이크로 프로세서
- Atmel 사의 AVR - ATmega328 탑재
- 하버드 구조의 8비트 RISC 단일 칩으로 구성
  - 하버드 구조: 명령을 불러오는 메모리와 데이터를 저장하는 메모리가 서로 다르다. 대신 CPU의 구조가 복잡하다.
  - 폰 노이만 구조: 모든 데이터를 메인 메모리에 저장하고 불러와 사용한다. 프로그램에 따라 병목현상이 발생할 수 있다.
  - RISC: 명령어 단순하고 세분화되어 있다.
  - CISC: 명령어가 복잡하지만 호출하는 수가 적다.

Figure 4-1: Harvard vs. von Neumann Block Architectures

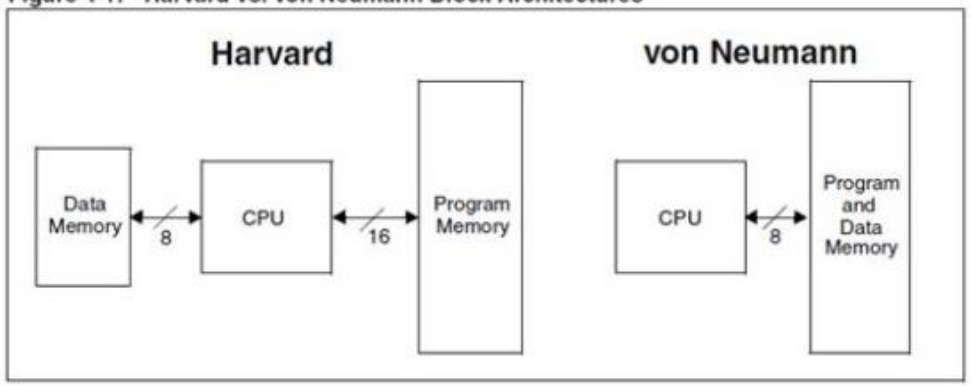


아두이노 구성 (1/4)

마이크로 컨트롤러(Micro Controller)

- 마이크로 컨트롤러: 하나의 칩에 중앙처리장치(CPU) 뿐만 아니라 일정 용량의 메모리와 입출력 인터페이스 까지 내장한 특수 목적용 마이크로 프로세서
- Atmel 사의 AVR - ATmega328 탑재
- 하버드 구조의 8비트 RISC 단일 칩으로 구성
  - 하버드 구조: 명령을 불러오는 메모리와 데이터를 저장하는 메모리가 서로 다르다. 대신 CPU의 구조가 복잡하다.
  - 폰 노이만 구조: 모든 데이터를 메인 메모리에 저장하고 불러와 사용한다. 프로그램에 따라 병목현상이 발생할 수 있다.
  - RISC: 명령어 단순하고 세분화되어 있다.
  - CISC: 명령어가 복잡하지만 호출하는 수가 적다.

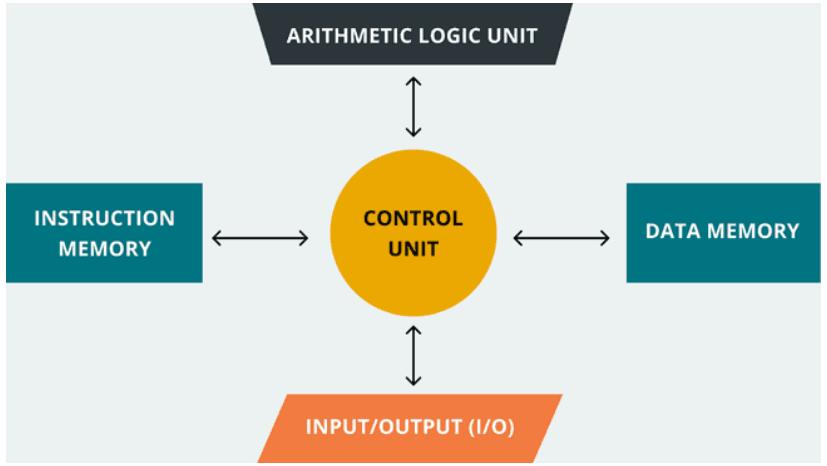
Figure 4-1: Harvard vs. von Neumann Block Architectures

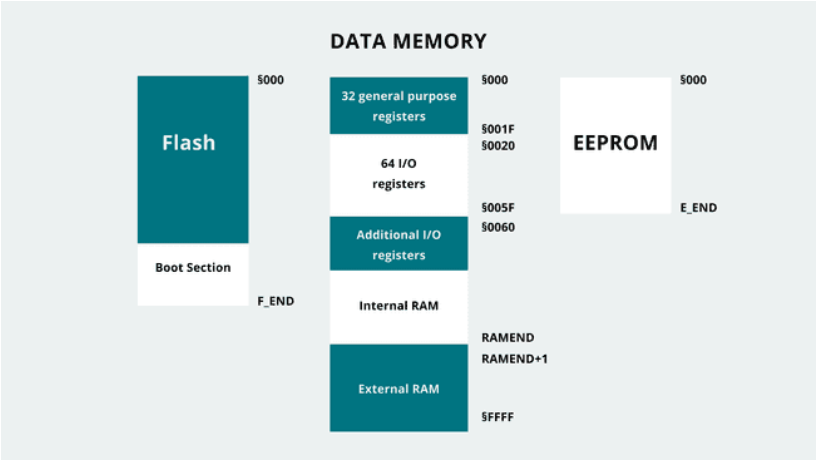


아두이노 구성 (2/4)

메모리(Memory)

- 아두이노의 메모리: SRAM, Flash, EEPROM
- Flash : 프로그램 메모리
- SRAM(Static Random Access Memory) : 데이터 메모리
- EEPROM(Electrically Erasable Programmable Read Only Memory)
    - 프로그램 실행 중에 중요한 데이터를 저장하고 값을 변경

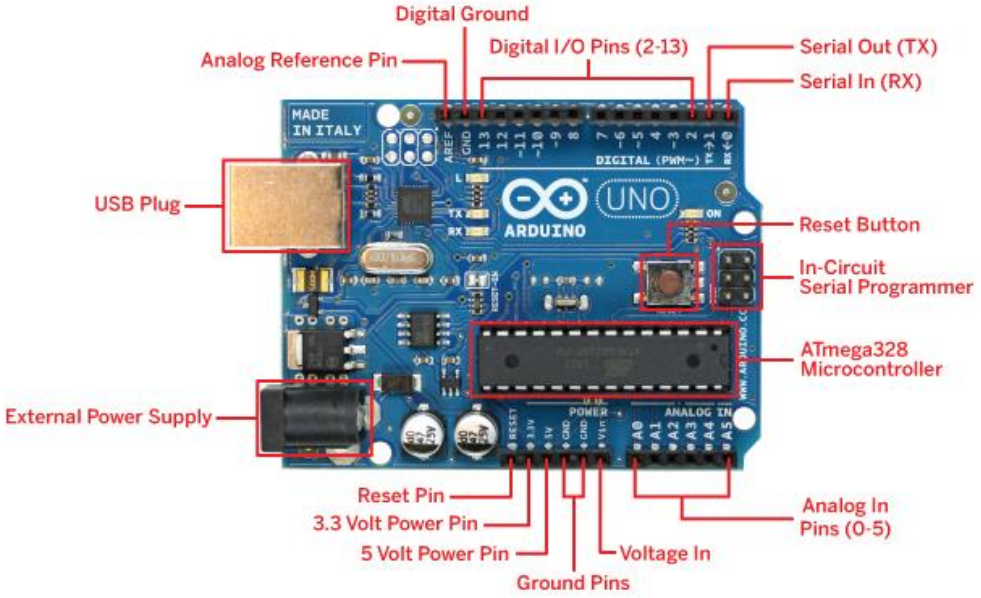




아두이노 구성 (3/4)

전압(Voltage)

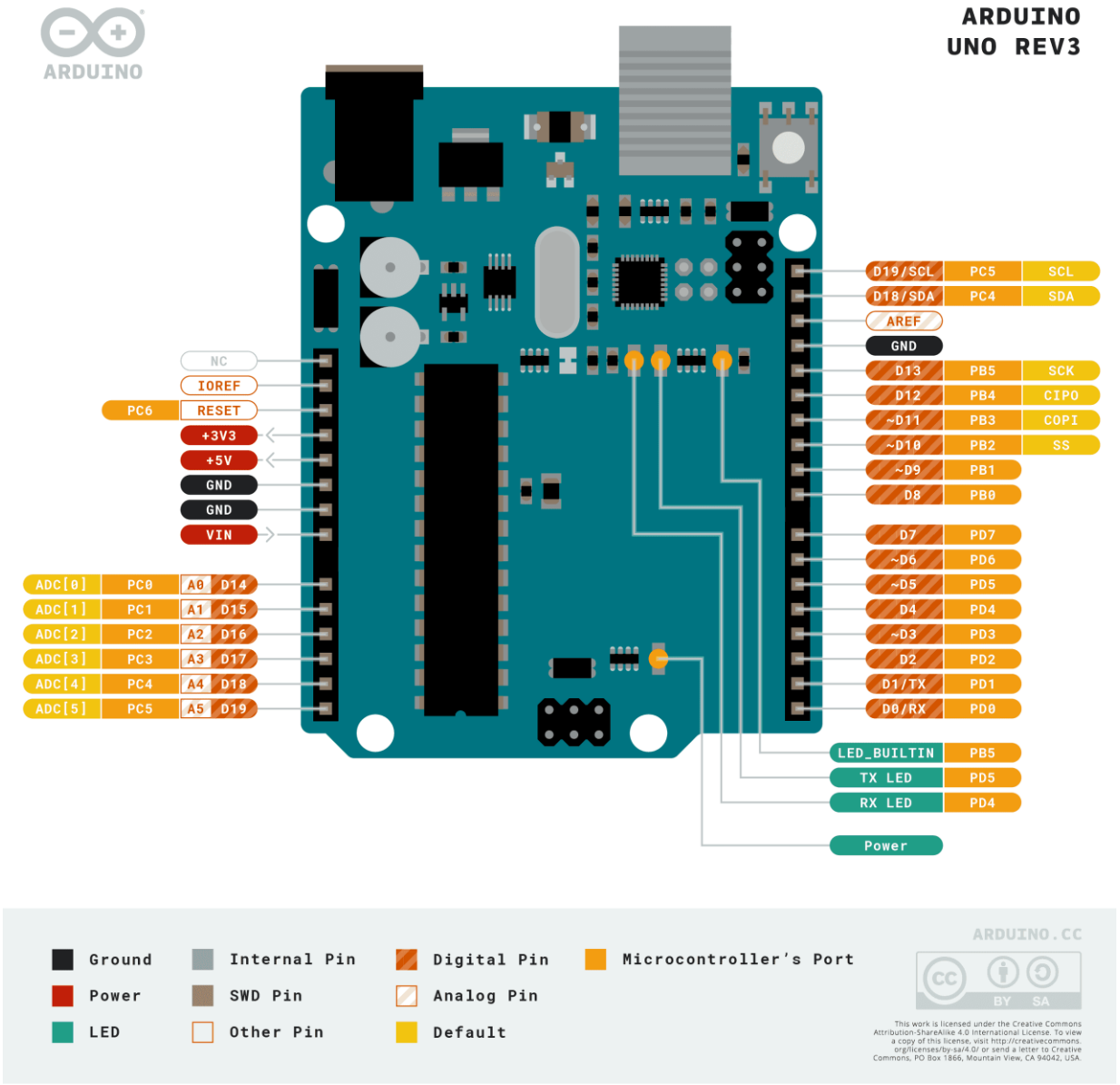
- 5V의 전압으로 동작
- 입력되는 전압은 6~12V
  - USB 케이블: 5V, 최대 500mA
  - 배럴잭 or Vin: -, 최대 1A
  - 혼용 시, 배럴잭의 전압이 6.6V 이하인 경우 USB로 입력되며, 초과하는 경우 배럴잭으로 입력



아두이노 구성 (4/4)

핀 헤더

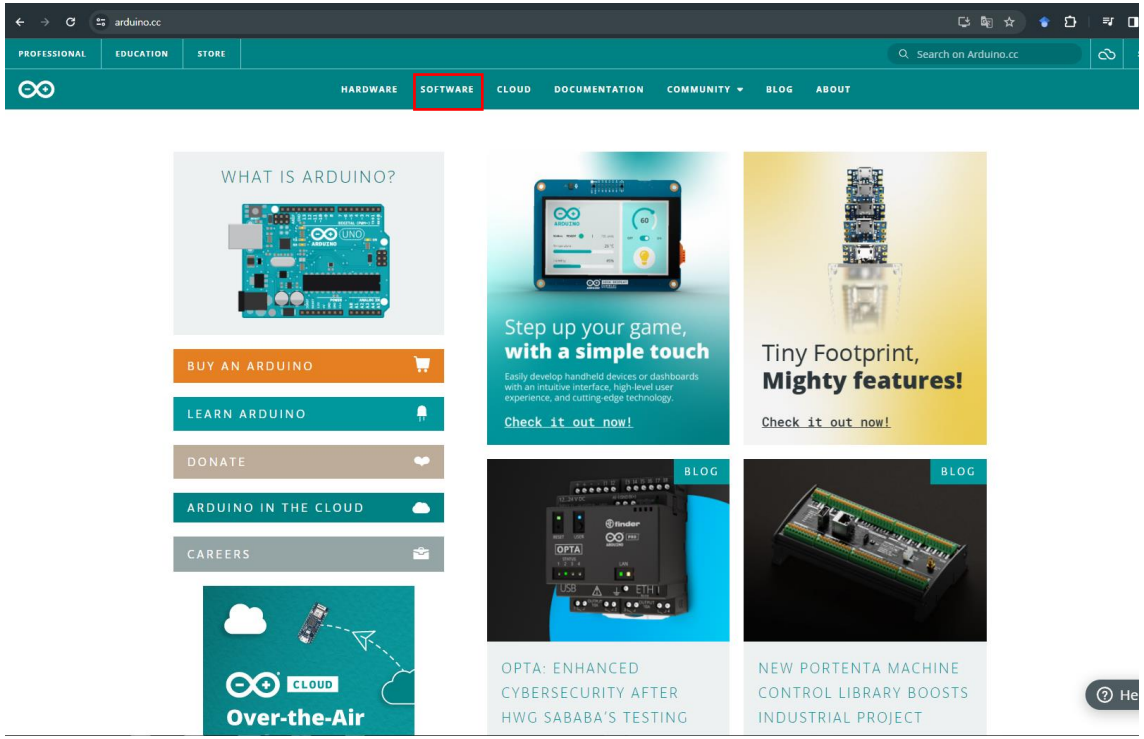
- 32개의 데이터 핀이 존재
  - 디지털 입출력 : 14개
  - 아날로그 입력: 6개
  - 아날로그 출력(PWM): 6개
  - 전원: 6개
  - RESET
  - IOREF, AREF: 기준 전압
  - SCL, SDA : 시리얼 통신
  - NC : not connected
- 각 핀 번호를 통해 데이터 교환

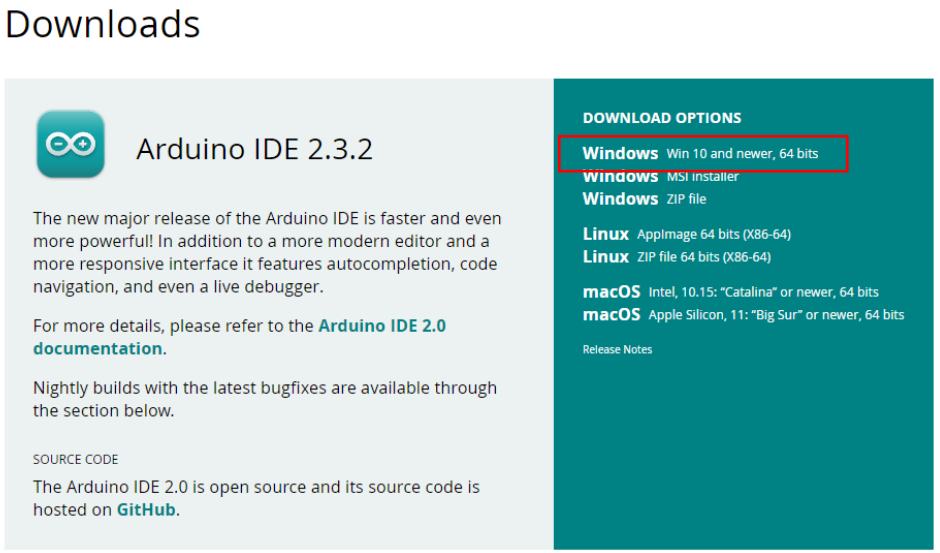


아두이노 IDE (1/4)

설치 방법

- Arduino.cc -> Software -> 버전 선택 -> Just download -> 설치 파일 실행 후 차례대로 진행 -> 프로그램 실행
- 설치 경로는 임의로 지정

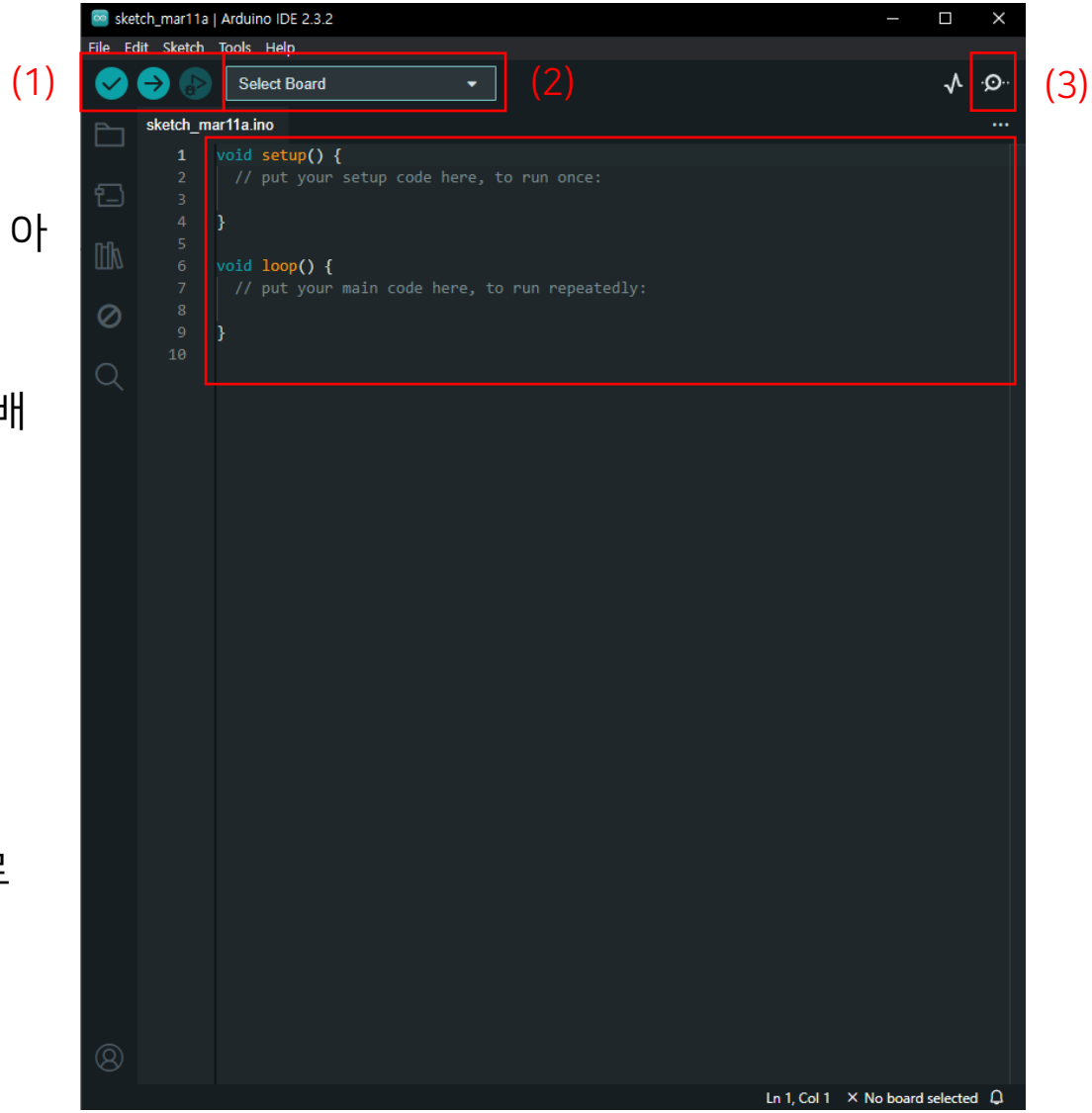




아두이노 IDE (2/4)

IDE 구조

- (1) 컴파일 및 업로드: 작성한 코드를 컴파일 하거나 연결된 아두이노 보드로 업로드 할 수 있다.
- (2) 보드 선택: 아두이노를 컴퓨터에 연결하면 아두이노에 배정된 포트로 연결할 수 있다.
- (3) 시리얼 모니터
- (4) 코드
    - void setup() : 아두이노의 사전 설정을 수행
    - void loop() : setup() 함수가 수행된 이후 반복적으로 실행되는 함수. 거대한 while() 문장



아두이노 IDE (3/4)

아두이노의 main() 함수

```
#include <Arduino.h>
int atexit(void (* /*func*/ )()) { return 0; }
// Weak empty variant initialization function.
// May be redefined by variant files.
void initVariant() __attribute__((weak));
void initVariant() { }
void setupUSB() __attribute__((weak));
void setupUSB() { }
int main(void)
{
        init();
        initVariant();

#if defined(USBCON)
        USBDevice.attach();
#endif
        setup();
        for (;;) {
                loop();
                if (serialEventRun) serialEventRun();
        }
        return 0;
}
```

아두이노 IDE (4/4)

시리얼 모니터

- 시리얼 통신을 수행하는 창.
- (1) - 컴퓨터에서 아누이노로 전송할 문자열을 입력하는 창
- (2) - 아두이노에서 컴퓨터로 전송된 문자열 출력창
- (3) - 전송하는 문자열의 끝에 추가하는 문자의 방식을 설정
- (4) - 시리얼 통신 속도를 조절하는 창

- 텍스트 기반으로 동작한다.
- (3) 항목은 다음과 같다.
- No Line Ending: 문자열 입력창에 입력한 문자만 입력한다.
- New Line : 문자열 끝에 '₩n' 문자를 입력한다.
- Carriage Return : 문자열 끝에 '₩r' 문자를 입력한다.
- Both NL & CR : 둘 다 입력한다.

아두이노 기본 클래스 함수 (1/3)

Serial

- 하드웨어 시리얼 포트를 통한 UART Universal Asynchronous Receiver Transmitter 시리얼 통신
- 헤더 파일 없이 사용 가능
- `HardwareSerial`의 유일한 객체로 미리 선언되어 있다

함수	begin	print / println / write
	UART 시리얼 통신 초기화	데이터 송신을 위한 함수 (print: 문자열 / write: 이진데이터)
함수원형	void Serial::begin(unsigned long baud) void Serial::begin(unsigned long baud, byte config)	size_t Serial::print(value, format)* size_t Serial::write(uint8_t ch)
매개변수	baud: 통신속도(보드레이트) config: 데이터 비트 수, 패리티, 정지 비트 설정	value: 출력값 format: 출력형식 ch: 출력할 바이트 단위 데이터
반환값	-	시리얼 포트로 출력된 바이트 수

아두이노 기본 클래스 함수 (2/3)

Serial

함수	available	peek
	시리얼 통신 버퍼에 데이터가 있는지 확인	수신 버퍼에서 1바이트를 읽어와 반환하는 함수
함수원형	int Serial::available()	int Serial::peek()
매개변수	-	-
반환값	시리얼 통신 수신 버퍼에 저장된 데이터의 바이트 수	시리얼 통신 수신 버퍼의 첫 번째 바이트 데이터 또는 -1

함수	read
	수신 버퍼에서 1바이트를 읽어와 반환하고 삭제하는 함수
함수원형	int Serial::read()
매개변수	-
반환값	시리얼 통신 수신 버퍼의 첫 번째 바이트 데이터 또는 -1

아두이노 기본 클래스 함수 (3/3)

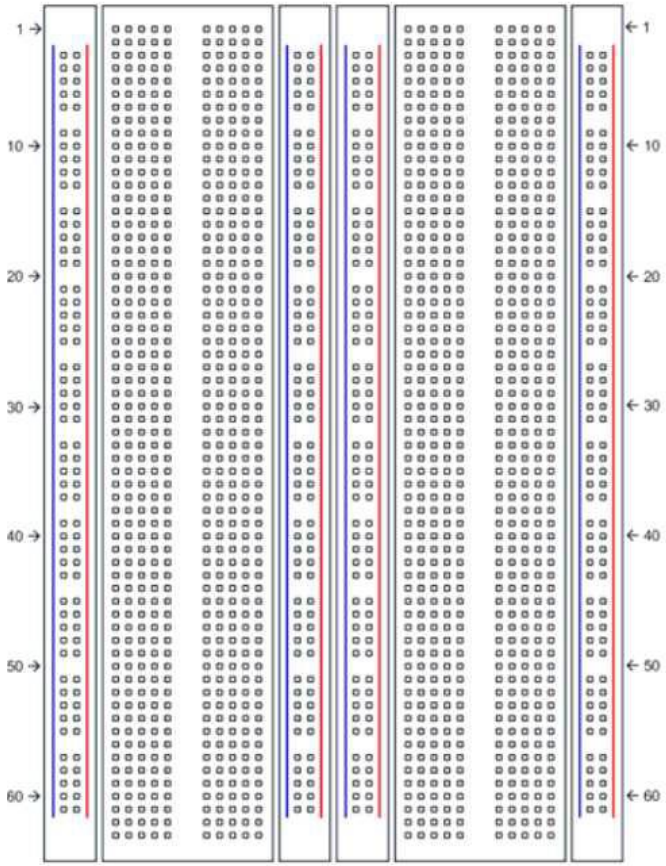
입출력 함수

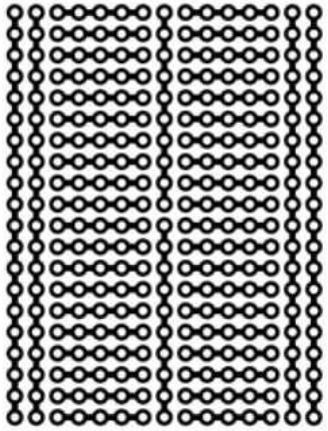
함수	pinMode	digitalWrite
	지정한 핀 번호를 입력 또는 출력으로 설정	비트 단위의 데이터를 출력
함수원형	void pinMode(uint8_t pin, uint8_t mode)	void digitalWrite(uint8_t pin, uint8_t value)
매개변수	pin: 설정하고자 하는 핀 번호 mode: OUTPUT, INPUT, INPUT_PULLUP 중 하나	pin: 핀 번호 mode: HIGH(1) 또는 LOW(0)
반환값	-	-

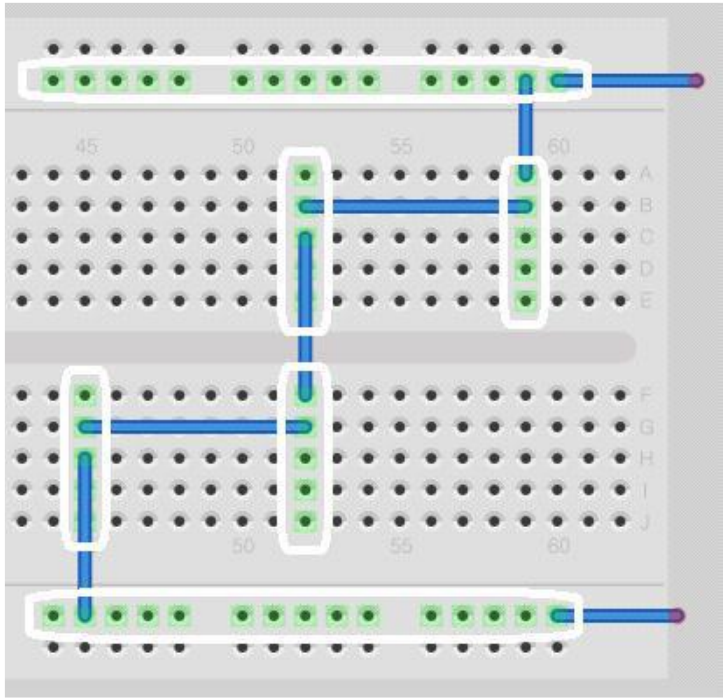
함수	delay	digitalRead
	지정한 밀리초(ms) 단위의 시간만큼 프로그램을 일시 정지	비트 단위의 데이터를 입력 받는 함수
함수원형	void delay(unsigned long ms)	void digitalRead(uint8_t pin)
매개변수	ms: 밀리초 단위의 지연 시간	pin: 핀 번호
반환값	-	HIGH(1) 또는 LOW(0)

기본 회로 (1/3)

브레드보드(Breadboard)

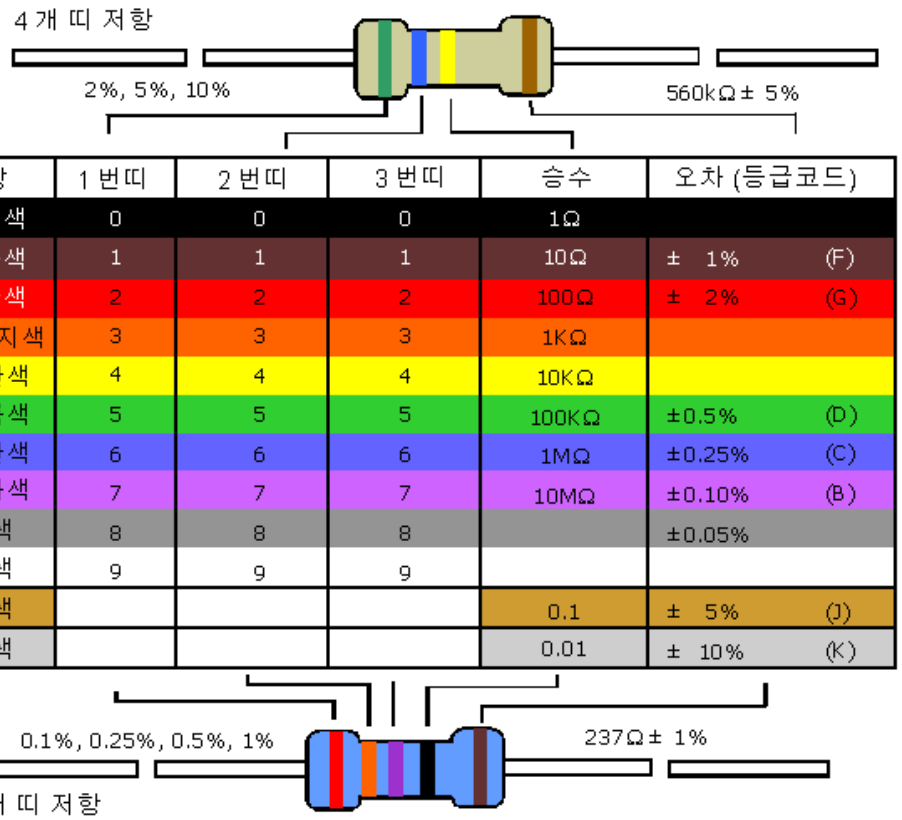




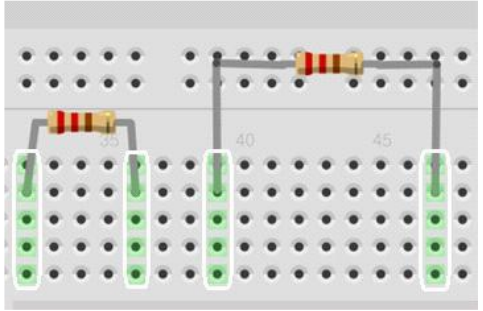


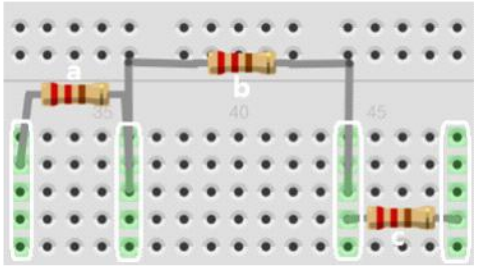
기본 회로 (2/3)

저항(Resistance)



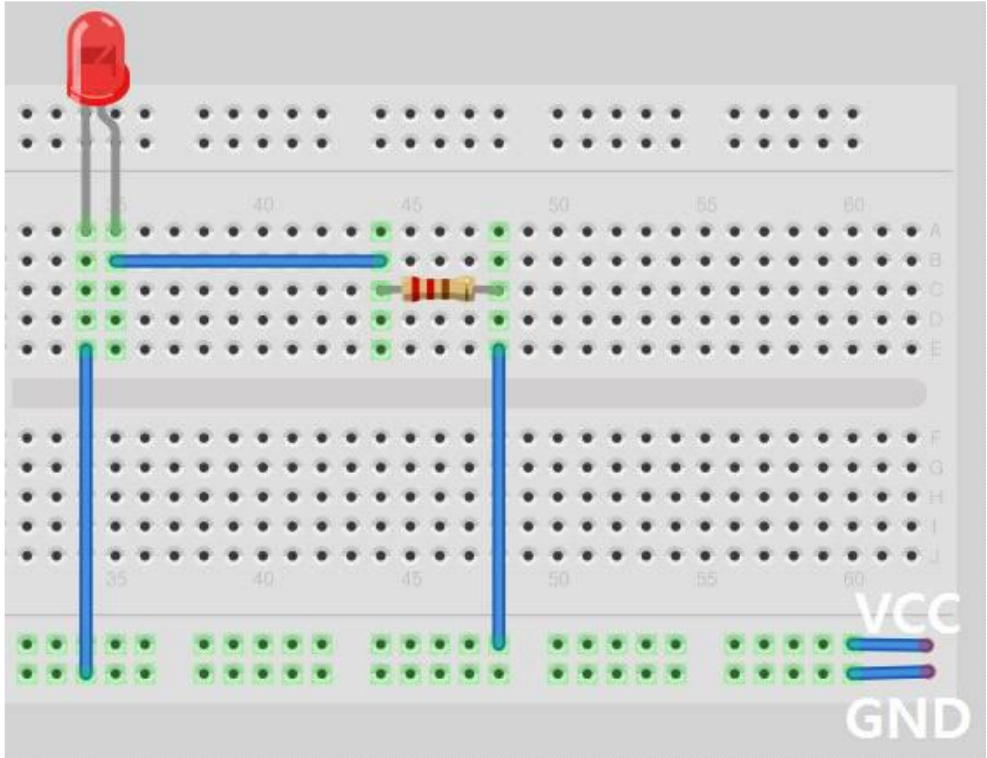
색상	1 번띠	2 번띠	3 번띠	승수	오차 (등급코드)	
검정색	0	0	0	1Ω		
고동색	1	1	1	10Ω	± 1%	(F)
빨간색	2	2	2	100Ω	± 2%	(G)
오랜지색	3	3	3	1KΩ		
노란색	4	4	4	10KΩ		
초록색	5	5	5	100KΩ	±0.5%	(D)
파란색	6	6	6	1MΩ	±0.25%	(C)
보라색	7	7	7	10MΩ	±0.10%	(B)
회색	8	8	8		±0.05%	
흰색	9	9	9			
금색				0.1	± 5%	(J)
은색				0.01	± 10%	(K)

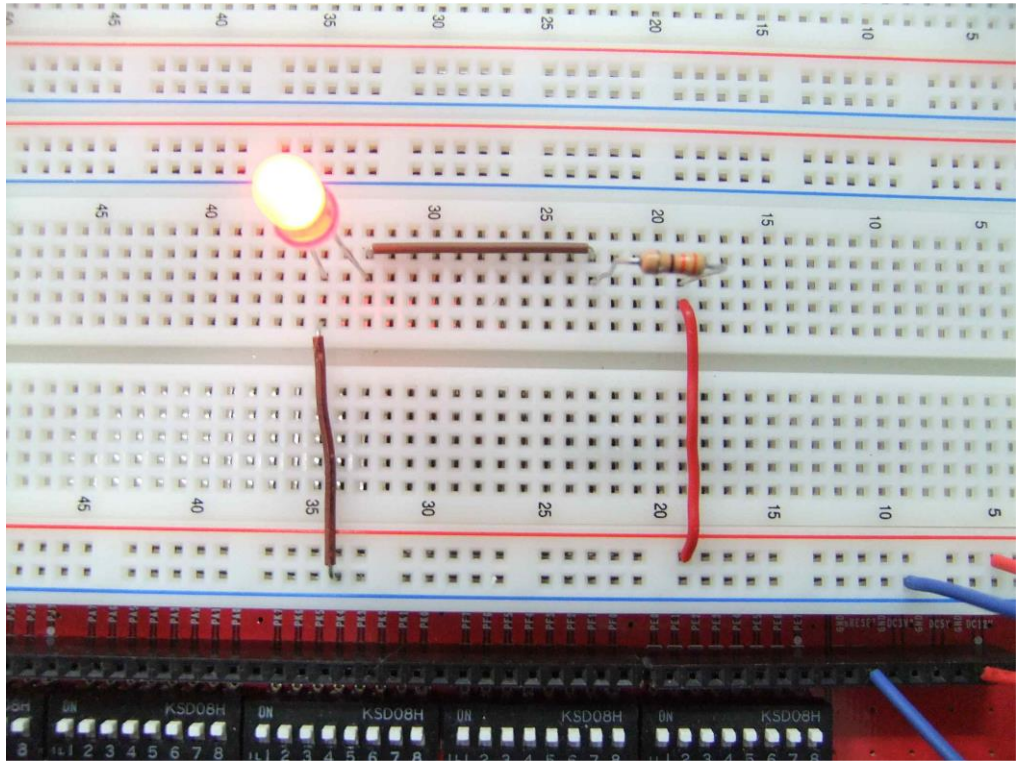




기본 회로 (3/3)

LED 점등하기





기본 예제 (1/3)

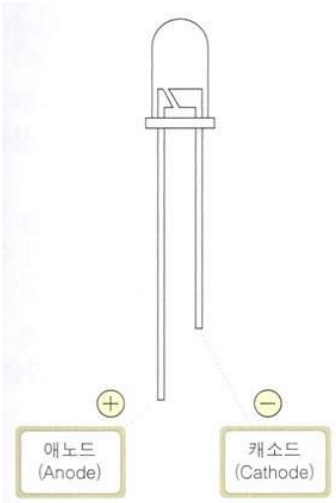
시리얼 통신

```
void setup() {
  Serial.begin(9600);
  Serial.println("input string:");
}
void loop() {
  if(Serial.available()){
      char ch = Serial.read();
      Serial.print(ch);
  }
}
```

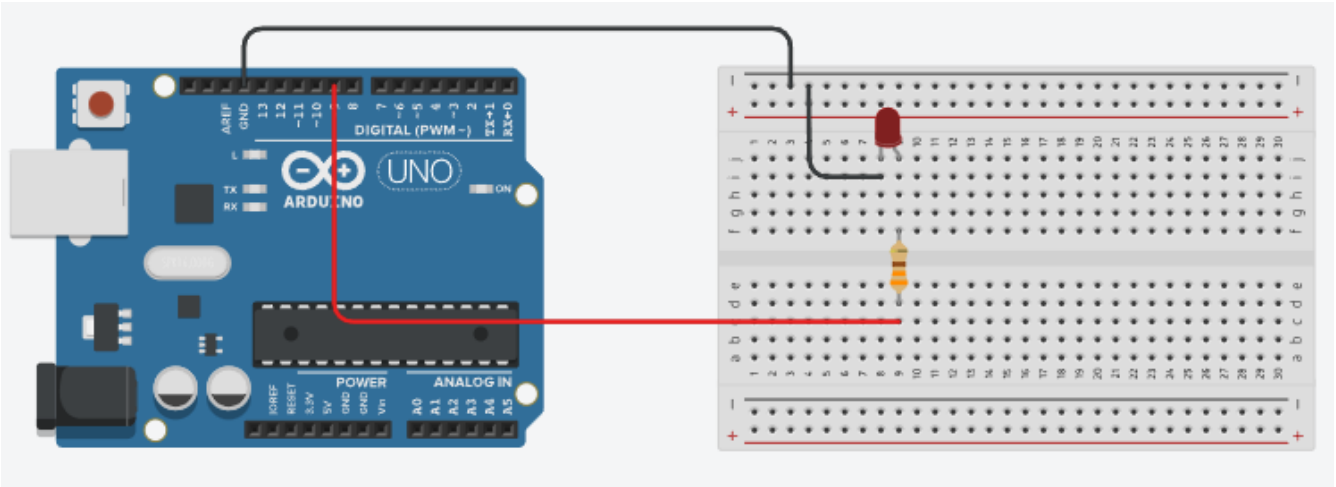
기본 예제 (2/3)

LED

- LED 핀의 (+)를 9번, (-)를 GND에 연결
- 9번 핀과 LED의 + 사이에 330옴 저항 연결




```
#define LED_PIN 9
void setup() {
   pinMode( LED_PIN, OUTPUT );
}
void loop() {
   digitalWrite(LED_PIN, HIGH);
   delay(1000);
   digitalWrite(LED_PIN, LOW);
   delay(1000);
}
```

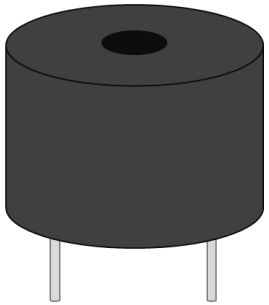


기본 예제 (3/3)

능동부저

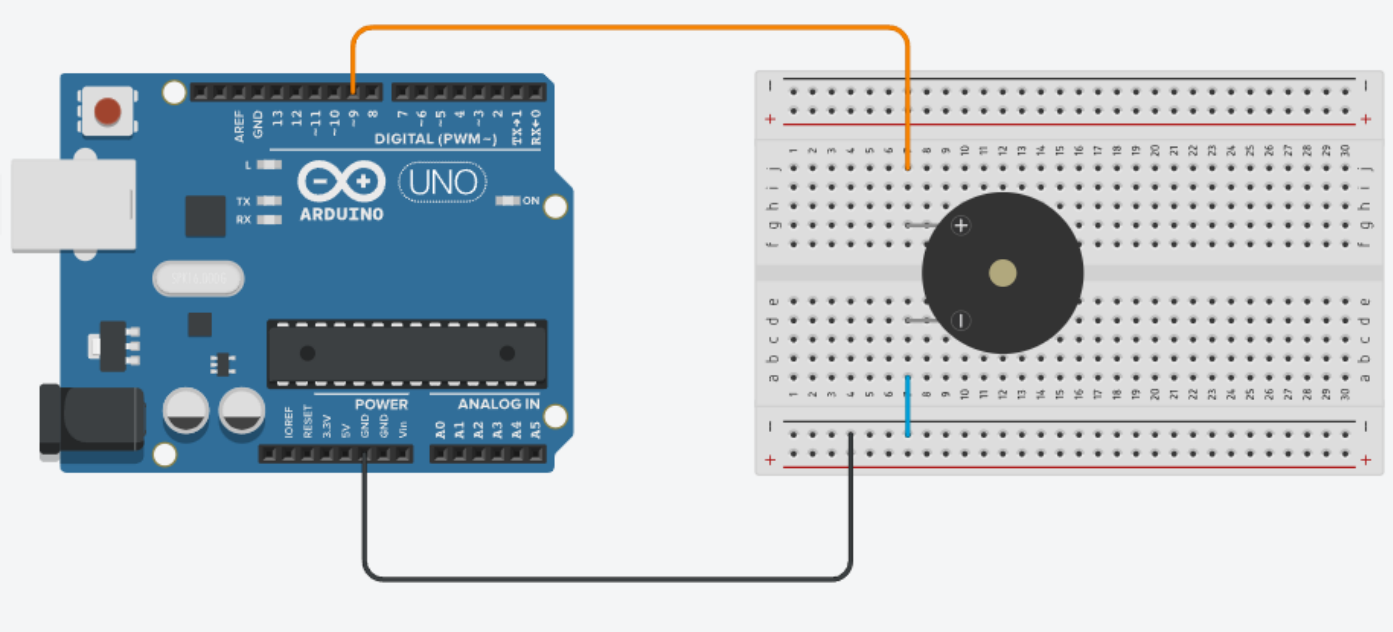
- 내부의 고정된 주파수를 출력하는 장치
- 단순한 음만 출력 가능

능동 부저



수동 부저

```
#define LED_PIN 9
void setup() {
    pinMode( LED_PIN, OUTPUT );
}
void loop() {
    digitalWrite(LED_PIN, HIGH);
    delay(1000);
    digitalWrite(LED_PIN, LOW);
    delay(1000);
}
```



과제 1

- 시리얼 모니터에 숫자 1을 입력하면 부저가 1초 동안 동작하도록 하시오.
- Serial.available() 함수는 시리얼 입력 버퍼에 저장된 데이터의 수를 반환한다.
- Serial.read() 함수는 버퍼에 저장된 첫번째 값을 읽어온다.
- 해당 문제는 조건문을 활용한다

고생하셨습니다

Thank you!